Unit 3 — Assignment: Build a RAG Chatbot

Objective

Build a small **Streamlit chatbot** that answers questions over a set of PDF documents. Internally, it must use the **RAG** pattern: retrieve the relevant chunks from the PDFs and let an LLM answer based on them. Every building block was covered in class; this assignment asks you to **glue them together** into a working app.

> **Important.** You must build this chatbot over **your own PDF content** — not the financial reports we used in class. Pick a corpus that matters to you: your own course notes, articles you've collected, a personal knowledge base, internship documentation, research papers, technical manuals — anything in PDF format. The topic is your call, but the corpus must be yours.

What you will learn

- **RAG pipeline**: chunking, embedding, indexing, retrieval, generation.
- **HuggingFace embeddings**: using local sentence-transformer models with LangChain.
- **Chroma vector store**: persisting and querying an index on disk.
- **Reranking**: improving retrieval quality with a cross-encoder.
- **Streamlit**: building a minimal chat UI with caching of heavy resources.

Required stack

- **Embeddings**: HuggingFace — `sentence-transformers/all-MiniLM-L6-v2` (fast, free, no API key).
- **Chat LLM**: Groq — `llama-3.1-8b-instant` or `llama-3.3-70b-versatile`.
- **Vector store**: Chroma (same as class).
- **UI**: Streamlit — `streamlit run app.py`.

What to build

1. Indexing script

Objective: load the PDFs, chunk them, embed them with HuggingFace, and store them in Chroma. Run it **once** — the index is then persisted to disk.

2. Streamlit app (app.py)

Your turn: build a minimal UI with:

- A text input for the question.
- A button to submit.
- An area that shows the LLM's answer.

3. Retrieval and answer flow

Objective: when the user clicks submit, the app must:

- Retrieve the most relevant chunks from Chroma.
- **Rerank** them with a cross-encoder (see next section).
- Pass the top reranked chunks + the question to Groq.
- Display the answer.

Mandatory: Reranking

After retrieving k chunks (e.g. `k=10`) from Chroma by similarity, you must **rerank** them with a cross-encoder before sending them to the LLM, and only pass the **top 3–5 reranked chunks** to the prompt.

Suggested library: `sentence-transformers` · `CrossEncoder` with the model `cross-encoder/ms-marco-MiniLM-L-6-v2`.

In your `README.md`, explain in **2–3 sentences** why reranking helps.

Optional (if you want to go further)

Pick **one** of these only if you have time after the mandatory part works:

- Try a different chunk size and explain whether the answers improved.
- Add a dropdown in the UI to filter by source or category before retrieving.
- Stream the LLM response token by token instead of waiting.

Deliverables

A zip or a Git repo with:

- Your code (the indexing script + `app.py`).
- A `requirements.txt`.
- A short `README.md` (one page max) with: how to run, which models you used, and **3 example questions you tested and the answers you got**.

> **Do not** include the PDFs, the Chroma database, or your API keys in the submission.

Hints

- `langchain-huggingface` is the package for HuggingFace embeddings. The first run downloads the model (a few hundred MB) — that's normal.
- Streamlit reruns the whole script on every click. Wrap your model and vector store loading in `@st.cache_resource` so they only load once.
- Don't rebuild the Chroma index on every app start — load the persisted index from disk after the first indexing run.

- Keep your `app.py` short. A working baseline is ~50 lines.